Agentic Multi-Modal LLMs for Software Comprehension: Structuring Code Summarization with Business Process Awareness

Srikanth Tamilselvam*, Ashita Saxena*
IBM Research, Bangalore
srikanth.tamilselvam@in.ibm.com, ashitasaxena@ibm.com

2025 IEEE International Conference on Software Services Engineering (SSE) | 979-8-3315-6789-7/25/$31.00 ©2025 IEEE | DOI: 10.1109/SSE67621.2025.00024

***Abstract*—Application Understanding task aims to help users comprehend an application's capabilities by systematically analyzing its artifacts. Ideally, such summaries should align with how the application is used in practice, highlighting essential workflows and functional modules in a structured manner. However, existing automated approaches often fall short of this expectation. Lack of application-specific background and domain knowledge limits the system's ability to present functionalities meaningfully. To address these challenges, we propose a novel agentic approach leveraging multi-modal LLMs that integrate code analysis, textual artifacts, and domain knowledge to identify key business flow entities—such as programs and tables—within a repository and infer application workflows. This work opens new avenues in LLM-guided software comprehension, bridging the gap between code-centric insights and high-level business process understanding.**

***Index Terms*—Application Understanding, program analysis, Code summarization, agents, domain alignment.**

I. Introduction

Summarizing large software applications is essential for understanding core functionalities, providing clear insights into system behavior, and improving maintainability while reducing onboarding time for new developers. While traditional automated code summarization has focused on function, program, or package levels [1], application-level understanding remains underexplored. Existing approaches from Hellman et al. [2] and tools like readme-generator*, rely on template-based methods that produce brief descriptions, which due to their lack of depth, offer limited practical utility. Recent advancements in large context LLMs, such as MS Copilot† and Cursor‡ etc. claim to provide repository-level explanations but struggle to align summaries with the natural business flow of applications. This limitation arises because these models primarily focus on structural and syntactic analysis while lacking domain-specific application knowledge. Functionality extraction has mainly been explored in monolith-to-microservices decomposition [3] [4], which relies on program dependencies. However, when key business logic is concentrated in a few programs, this approach can lead to oversized macroservices instead of meaningful functional grouping. Consider the business summary request for DayTrader§ illustrated in Figure 1. Github Copilot, with Claude Sonnet 3.5 fails to detect the logical business workflow of the application, because of which there is a) **lack of adequate abstraction** - Grouping of key functionalities like Quote lookup and trade operations, listing of non domain capabilities like performance related features and b) **lack of natural order** - stock trading is discussed before authentication, market summary is listed after trading. To the best of our knowledge, no prior work has focused on abstracting functionalities from code and text artifacts like README and structuring them in alignment with real-world business workflows. Therefore in this work we introduce a novel approach for business workflow-aware application summarization, ensuring that summaries are logically structured, domain-aligned, and reflective of actual end-user workflows.

*These authors contributed equally to this work.
*https://pypi.org/project/readme-generator/
†https://copilot.microsoft.com/
‡https://www.cursor.com/

II. Methodology

Our approach to business workflow-aware application summarization follows a multi-step pipeline that integrates application entry points and textual artifacts to generate structured, functionally coherent summaries as illustrated in Figure 2.

A. Input Sources

We use two primary inputs:

- **Entry Points:** Interfaces exposed to interact with the backend functionalities. We use Codellm-Devkit [5] to identify these entry points. For instance, in the DayTrader application, `TradeAppServlet` extends `HttpServlet` and serves as an entry point for key trading capabilities.
- **Textual Artifacts:** Documentation such as `README.md`, which may describe business services and workflows.

B. Entry Point Sufficiency Verification

A Verifier LLM assesses whether the identified entry points classes are sufficient to infer business services by evaluating their coverage of high-level functionalities. If found insufficient, the input expands to include all classes. For DayTrader, the Verifier Llama-3.3-70B¶ considered 45 entry point classes and confirms sufficiency.

§https://github.com/WASdev/sample.daytrader7/
¶https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct

979-8-3315-6789-7/25/$31.00 ©2025 IEEE
DOI 10.1109/SSE67621.2025.00024

Authorized licensed use limited to: Mukesh Patel School of Technology & Engineering. Downloaded on September 09,2025 at 14:38:15 UTC from IEEE Xplore. Restrictions apply.

TABLE I
EVALUATION RESULTS OF THE PROPOSED WORKFLOW-AWARE SUMMARIZATION SYSTEM

Application	Are Entry Points Sufficient?	Generated Clusters of Business Capabilities	Extracted Order of Business Services	Cluster Correctness & Mapping Accuracy	Work Flow Correctness
DayTrader [Trading App, #classes = 111]	Yes	Login Functionality, Stock Trading, Stock Quote Lookup, Session Management, Portfolio Management	Login Functionality → Portfolio Management → Stock Quote Lookup → Stock Trading	Complete Match	Fully Aligned
PlantsByWebSphere [Plant Inventory App, #classes = 36]	No	Account Management, Shopping and Order Management, Inventory and Backorder Management, Supplier Management, Help and Utilities, Image and Populate	Account Management → Shopping and Order Management → Inventory and Backorder Management → Supplier Management	Complete Match	Fully Aligned
Acme-Air [Airline App, #classes = 38]	Yes	User Management, System Configuration and Monitoring, Flight Management, Booking Management, Data Analytics	User Management → Flight Management → Booking Management	Complete Match	Fully Aligned
PetClinic [Veterinary Clinic Management System,#classes = 30]	Yes	Pet Management, Owner Management, Veterinarian Management, Error Handling	Owner Management → Pet Management → Veterinarian Management	Partial Match	Fully Aligned

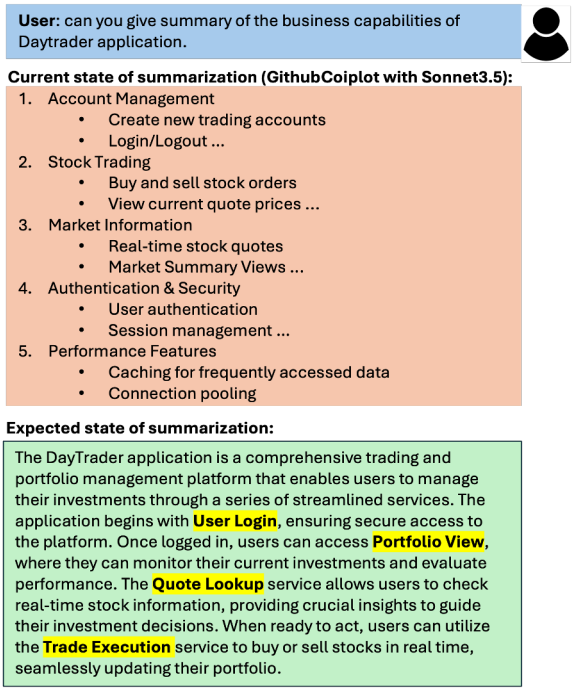

Fig. 1. Comparison of business summary generated by GitHub Copilot with Claude Sonnet 3.5 and expected summary, highlights inadequate abstraction of key capabilities and incorrect sequencing of business operations.

C. Textual Artifact Analysis

A Readme Analyzer LLM processes `README.md` to extract business services and workflow descriptions, storing them in a structured JSON format. If no relevant information is found, the fields *capabilities and workflow* remain null. For DayTrader, Llama-3.3-70B identifies business services like *login*, *view portfolio*, *lookup stock quotes*, and *buy/sell stocks*.

D. Business Capability Identification

A LLM clusters entry points (or all classes, if expanded) into business services based on:

- Functional similarity among entry points.
- Business service descriptions from documentation.

This ensures meaningful clusters aligned with distinct functionalities rather than low-level operations. For DayTrader, clusters include *Login*, *Stock Trading*, *Portfolio Management*, *Session Management*, and *Stock Quote Lookup*. Non-functional clusters like *Session Management* are identified but are ignored for business capabilities [‖] summaries.

E. Business Capability Organization

A Business Capability Organizer LLM arranges clusters into a logical user journey by:

- Aligning with extracted workflow information.
- Filtering out non functional clusters.

This ensures a structured, real-world flow in the final summary. For DayTrader this results in the following order: *Login Functionality* → *Portfolio Management* → *Stock Quote Lookup*

[‖]capabilities, functionalities, services are used interchangeably

Authorized licensed use limited to: Mukesh Patel School of Technology & Engineering. Downloaded on September 09,2025 at 14:38:15 UTC from IEEE Xplore. Restrictions apply.

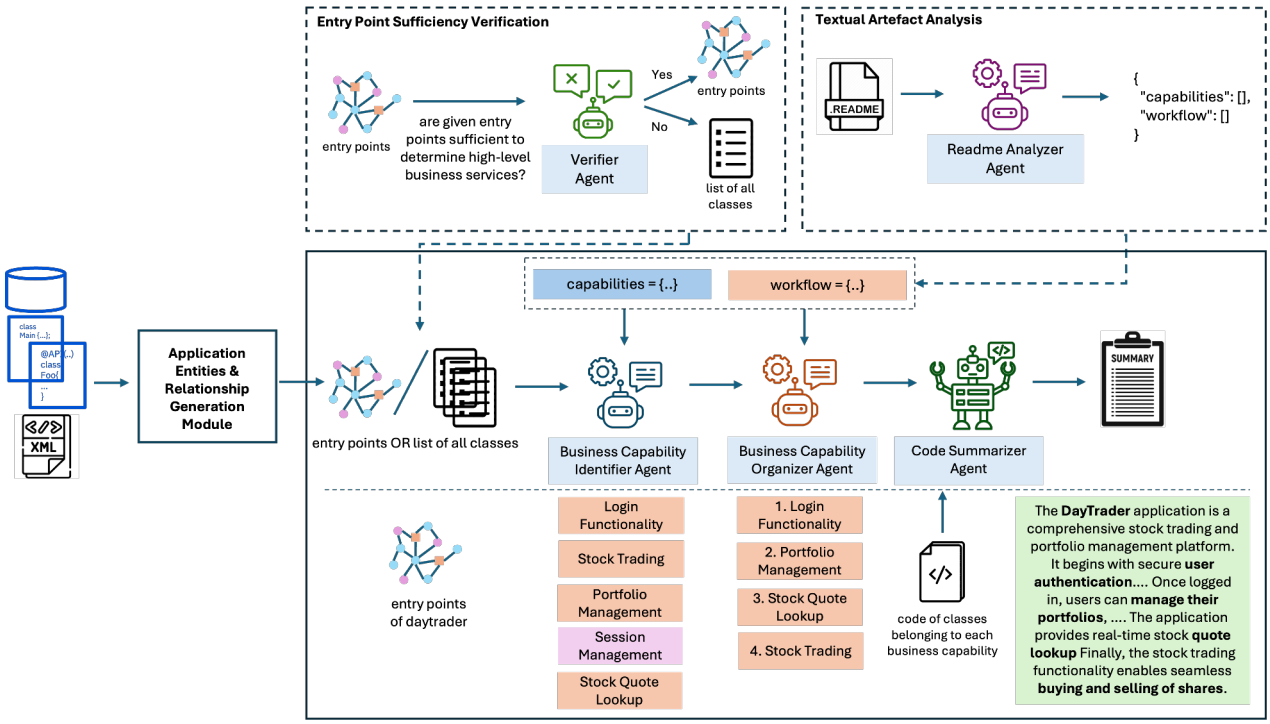

Fig. 2. LLM based business workflow-aware application summarization steps leveraging code and document analysis

→ *Stock Trading*. Notably, the non-essential cluster *Session Management* is filtered out in this step.

F. Summarization of Business Service Clusters

Each business service cluster is summarized using the Granite 3.1 model**. The model processes full code from all classes within a cluster to generate a concise, functionally meaningful summary. The LLM then integrates these summaries into a comprehensive, well-structured application overview.

G. Final Application Summary

The resulting summary:

- Abstracts technical details into high-level business functionalities.
- Follows a logical, user-centric order.
- Provides a structured overview of the application's capabilities.

III. Experiments and Results

To evaluate our methodology, we chose four publicly available web-based Java monolith applications: `DayTrader`*, `PlantsByWebSphere`†, `Acme-Air`‡, and `PetClinic`§. Each application was processed through our multi-step workflow-aware summarization pipeline.

**https://huggingface.co/ibm-granite/granite-3.1-8b-instruct
*https://github.com/WASdev/sample.daytrader7
†https://github.com/WASdev/sample.plantsbywebsphere
‡https://github.com/acmeair/acmeair
§https://github.com/spring-projects/spring-petclinic

We employed a subject matter expert (SME) with eighteen years of java development experience and actively works on application modernization to understand the four applications and confirm if the output business services are identified correctly and confirm that their order reflect the end user usage. Table I captures the application details, outcomes of our pipeline and SME feedback (last two columns). Our LLM based solution was able to accurately identify core business services in three out of four applications. Only for petclinic, it missed identifying `Schedule Visits` service but got the workflow alignment completely correct for all the applications.

IV. Conclusion and FuturWorks

Our proposed LLM agents leverage multi-modal inputs, such as code and documents, to implement a workflow-aware summarization approach that effectively extracts and organizes business services, aligning them with natural end-user workflows. Future work includes improving service identification through finer-grained domain adaptation, expanding evaluation to enterprise applications, and extending support to other programming languages.

References

[1] C. Zhang, J. Wang, Q. Zhou, T. Xu, K. Tang, H. Gui, and F. Liu, "A survey of automatic source code summarization," *Symmetry*, vol. 14, no. 3, p. 471, 2022.

[2] J. Hellman, E. Jang, C. Treude, C. Huang, and J. L. Guo, "Generating github repository descriptions: A comparison of manual and automated approaches," *arXiv preprint arXiv:2110.13283*, 2021.

Authorized licensed use limited to: Mukesh Patel School of Technology & Engineering. Downloaded on September 09,2025 at 14:38:15 UTC from IEEE Xplore. Restrictions apply.

[3] A. Mathai, S. Bandyopadhyay, U. Desai, and S. Tamilselvam, "Monolith to microservices: Representing application software through heterogeneous graph neural network."

[4] U. Desai, S. Bandyopadhyay, and S. Tamilselvam, "Graph neural network to dilute outliers for refactoring monolith application," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 35, no. 1, 2021, pp. 72–80.

[5] R. Krishna, R. Pan, R. Pavuluri, S. Tamilselvam, M. Vukovic, and S. Sinha, "Codellm-devkit: A framework for contextualizing code llms with program analysis insights," *arXiv preprint arXiv:2410.13007*, 2024.

Authorized licensed use limited to: Mukesh Patel School of Technology & Engineering. Downloaded on September 09,2025 at 14:38:15 UTC from IEEE Xplore. Restrictions apply.